

A COMPUTATIONAL MODEL OF AESTHETIC JUDGMENT

Memory, the Inverted-U, and Control

Instance-based learning as a substrate for aesthetic preference — and for an agent that tries to steer it

● Mark G. Orr · pyactup / Algorithm A4 & Algorithm C

Why do we like what we hear?

Berlyne's inverted-U hypothesis

- Aesthetic response is theorized to rise, peak, then fall as a stimulus's complexity (or novelty, or arousal potential) increases — too simple is boring, too complex is overwhelming.
- The peak marks a moderate, intermediate level of complexity: the “Wundt curve.”
- Widely cited as fading from relevance since the 1980s in favor of newer, more ecologically-grounded theories.

87.7%

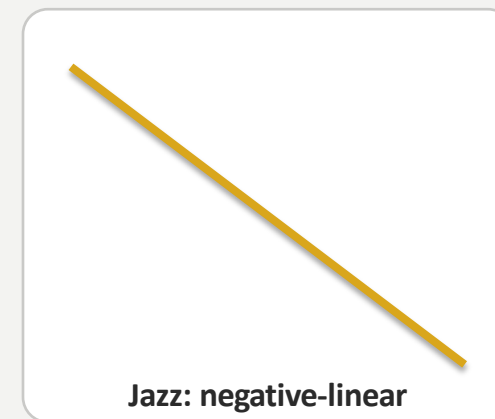
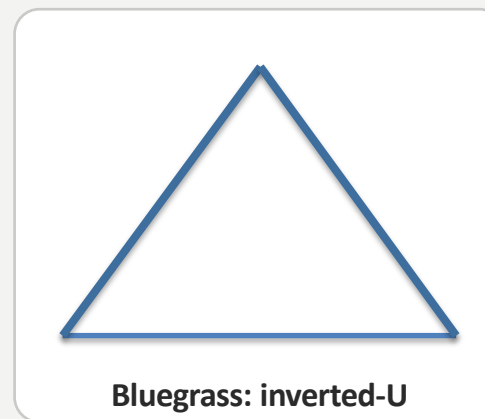
of 57 music-preference studies reviewed (1901–2015)
produced results compatible with some segment of the
inverted-U

*The model has been declared “defunct” less because it fails to
replicate, and more because research interest moved
elsewhere.*

The inverted-U isn't universal

Orr & Ohlsson (2001): jazz vs. bluegrass

- Non-musicians rated complexity and liking of short jazz and bluegrass improvisations, created by professional musicians at five controlled complexity levels.
- Bluegrass: strong, clean support for the inverted-U — both the ascending and descending legs of the curve held up.
- Jazz: only the descending leg was supported. A negative-linear relationship (simpler is always better) fit the jazz data just as well.
- Conclusion: the shape of the complexity–liking relationship can depend on musical style, not just on the collative variable itself.



Any model that claims to explain aesthetic judgment in general needs a way for the shape of the preference curve to depend on context — not just be a fixed formula.

Expertise changes the picture

Orr & Ohlsson (2005): does the inverted-U hold for experts?

- Professional jazz and bluegrass musicians (10+ years training, working professionals) rated the same style of stimuli as the 2001 study.
- Result: no inverted-U for either expert group, on either style. Complexity barely predicted liking at all for the jazz musicians, and only weakly (and linearly) for the bluegrass musicians on their own style.
- Comparing no-training, moderately-trained, and expert listeners: the variance in liking explained by complexity shrank as expertise grew.
- Proposed explanation: experts develop other, learned aesthetic criteria (timbre, phrasing, harmony) that come to matter more than raw complexity.

THE PATTERN

no training → moderate training → expert

strength of the complexity–liking link

 novice $R^2 \approx .56$

 moderate $R^2 \approx .33$

 expert $R^2 \approx .10$

(jazz-improvisation data, illustrative)

What a model needs to explain

Two open threads from the literature, one shared answer

Context-dependence

The curve's shape (or whether it appears at all) depends on the musical style being judged.

Experience-dependence

The curve weakens as the listener's accumulated experience with the domain grows.

Both are, at bottom, statements about accumulated exposure history — which style of song you've heard before, and how much of it. An instance-based learning model, where every exposure leaves a trace in memory, has this kind of history-sensitivity built in from the start, rather than needing to be added on top of a fixed formula.

Memory as the basis for aesthetic judgment

- Each exposure to a song is encoded as one memory chunk (instance-based learning, ACT-R / pyactup).
- Anderson's rational analysis of memory: a chunk's activation reflects how statistically predictable that item has been in the environment.
- The model's core move: compare the activation memory predicts for the next song against the activation of the song actually heard.
- That gap — predicted vs. actual activation — is the raw material for aesthetic judgment: “aesthetic_basis.”

ALGORITHM A

```
1.  retrieve()  →  predicted_activation  
  
2.  activation lookup for the song heard  
    →  actual_activation  
  
3.  aesthetic_basis = | predicted -  
    actual |
```

Chunks currently carry two slots: a song identifier and a value (evaluation itself). Aesthetic_basis is only defined once a song has a chunk of its own — i.e. from its second exposure onward.

Algorithm A4: building the inverted-U in

- A4 keeps a running average of `aesthetic_basis` over the whole simulation — cumulative, or a moving window — as a live estimate of “how surprising has this environment been lately.”
- That average, scaled by a free parameter γ (gamma), sets r .
- `evaluation` is a parabola in `aesthetic_basis`, anchored at 0, peaking at $x = r/2$, and returning to 0 at $x = r$ — an inverted-U, literally.
- The song's evaluation is then learned as a new memory chunk, feeding back into future predictions.

ALGORITHM A4

```
r =  $\gamma$  · time_averaged_aesthetic_basis
```

```
evaluation = 1 - (2 · x / r - 1) 2
```

```
where x = aesthetic_basis (this trial)
```

```
peak at x = r/2, zero at x = 0 and  
x = r
```

$\gamma = 1.0$ reduces A4 to Algorithm A3 exactly (verified row-for-row under zero noise). γ is left as an explicit tuning knob, not a fixed constant.

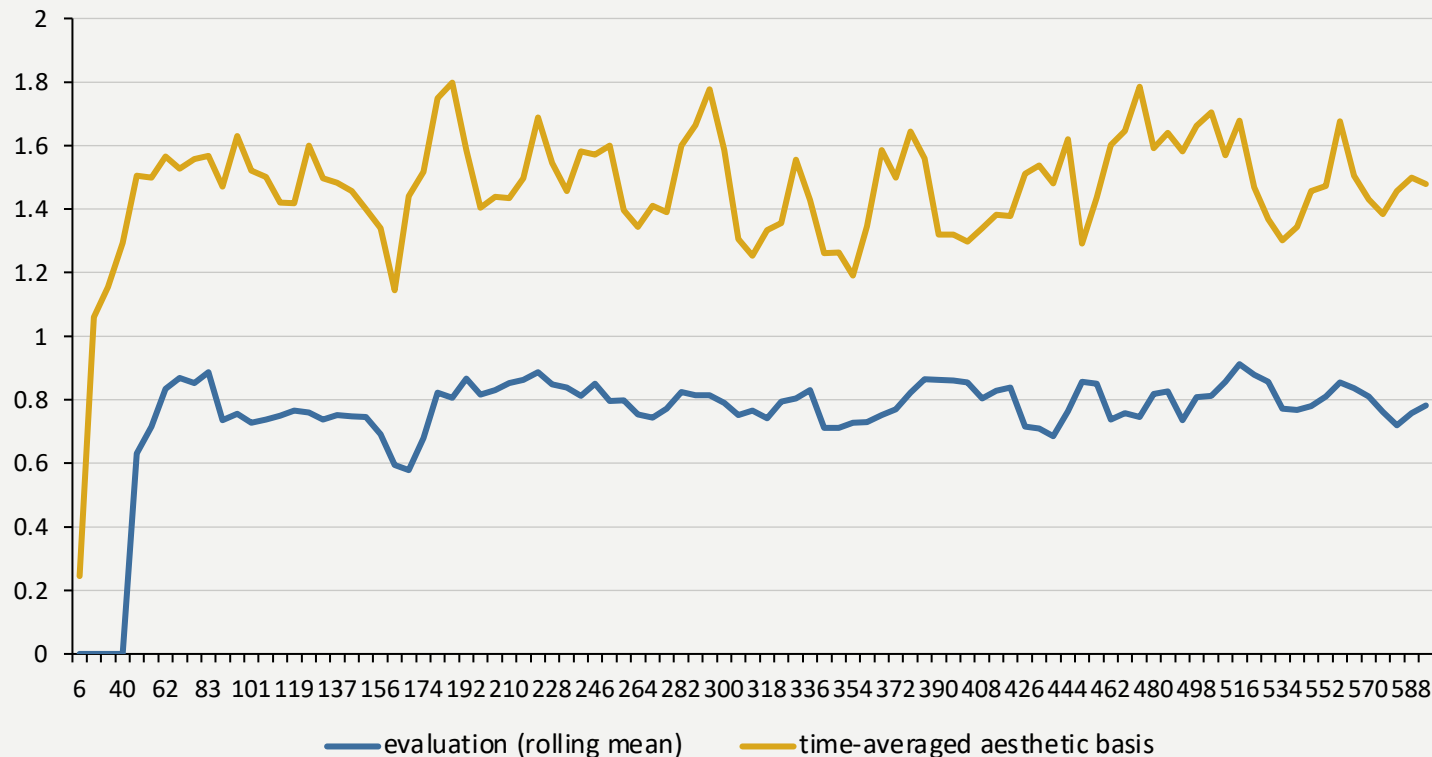
A passive listening baseline

Method

- 100 songs, each with a fixed complexity drawn once — a stationary, “ergodic” environment.
- Songs are drawn i.i.d. with replacement, uniformly at random, over thousands of exposures — no controller, no listener choice.
- Each repeat exposure runs Algorithm A4: `aesthetic_basis` → running average `r` → evaluation → a new `{song_id, evaluation}` chunk is learned.
- Two things are tracked over time: evaluation itself, and A4's own live `time_averaged_aesthetic_basis` — does the environment's memory-eye-view of “how surprising” settle down?

Results: a fast rise, then a fluctuating plateau

Design 5: evaluation and running surprise over 600 exposures



30 songs · 600 exposures · noise = 0.25 · $\gamma = 2.0$

0.90

median evaluation

1.47

typical time-averaged basis (last 300)

The time-averaged basis rises quickly (within roughly the first 80 exposures) from near 0 to about 1.5, then fluctuates around a plateau rather than smoothly settling — window mode tracks recent surprise, not the whole history. Combined with gamma=2's wider forgiving zone, evaluation stays consistently high even under random exposure.

Try it yourself

Live animated playback of a real Design 5 run

Watch the left-anchored evaluation parabola reshape trial-by-trial as r drifts, with a marker landing exactly on the curve at each trial's (aesthetic_basis, evaluation) — play / pause / scrub controls, synced against evaluation-over-time.

mark-orr.github.io/aesthetic-memory-model/design5-demo-2.html

From passive to active: a control problem

Design 5's environment picks songs uniformly at random. What if, instead, something tried to pick songs to maximize the memory agent's own evaluation?

1

Algorithm C1 — evaluation only

The controller sees only the evaluation number it gets back after playing a song. No access to the memory agent's internals at all.

2

Algorithm C2 — privileged access

Same controller, plus one extra signal: the agent's own live r (where the inverted-U peak currently sits).

Staged deliberately: the first-order controller establishes a baseline; the second-order controller isolates exactly what “knowing the target” buys you.

Two controllers, one bandit

Both controllers are a non-stationary multi-armed bandit

- Each song is an “arm.” Reward = the evaluation received the last time that song was played.
- Sliding-window UCB, not plain UCB: the reward for a given song genuinely drifts over time as memory and r evolve, so estimates are built from a recent window of plays, not the whole history.
- C1 stops there — pure evaluation-driven exploration/exploitation.

C2's EXTRA STEP: CHANGE-POINT RESET

- C2 also watches r each trial.
- If r moves by more than a threshold since the last trial, C2 wipes its bandit statistics and re-explores immediately.
- C1 has no such signal — it can only catch up gradually, as stale rewards age out of its own sliding window.
- This gives a concrete, testable prediction: C2 should re-adapt faster after the target moves, especially during the early transient.

The control study

Method

- Same fixed song pool as Design 5 — the bandit controller replaces the environment's random draw entirely, nothing else about the memory agent changes.
- Design 5 and both controllers are run with every model parameter held identical (below), so the only thing that differs between conditions is how the next song gets chosen.

MATCHED PARAMETERS — ALL THREE CONDITIONS

```
num_songs = 30          seed = 42
num_exposures = 600      decay = 0.5
gamma = 2.0    time_avg_mode = window (w=20)
noise = 0.25    (see tradeoff note, right)
```

C1 & C2 only: `bandit_window=20, bandit_c=2.0`

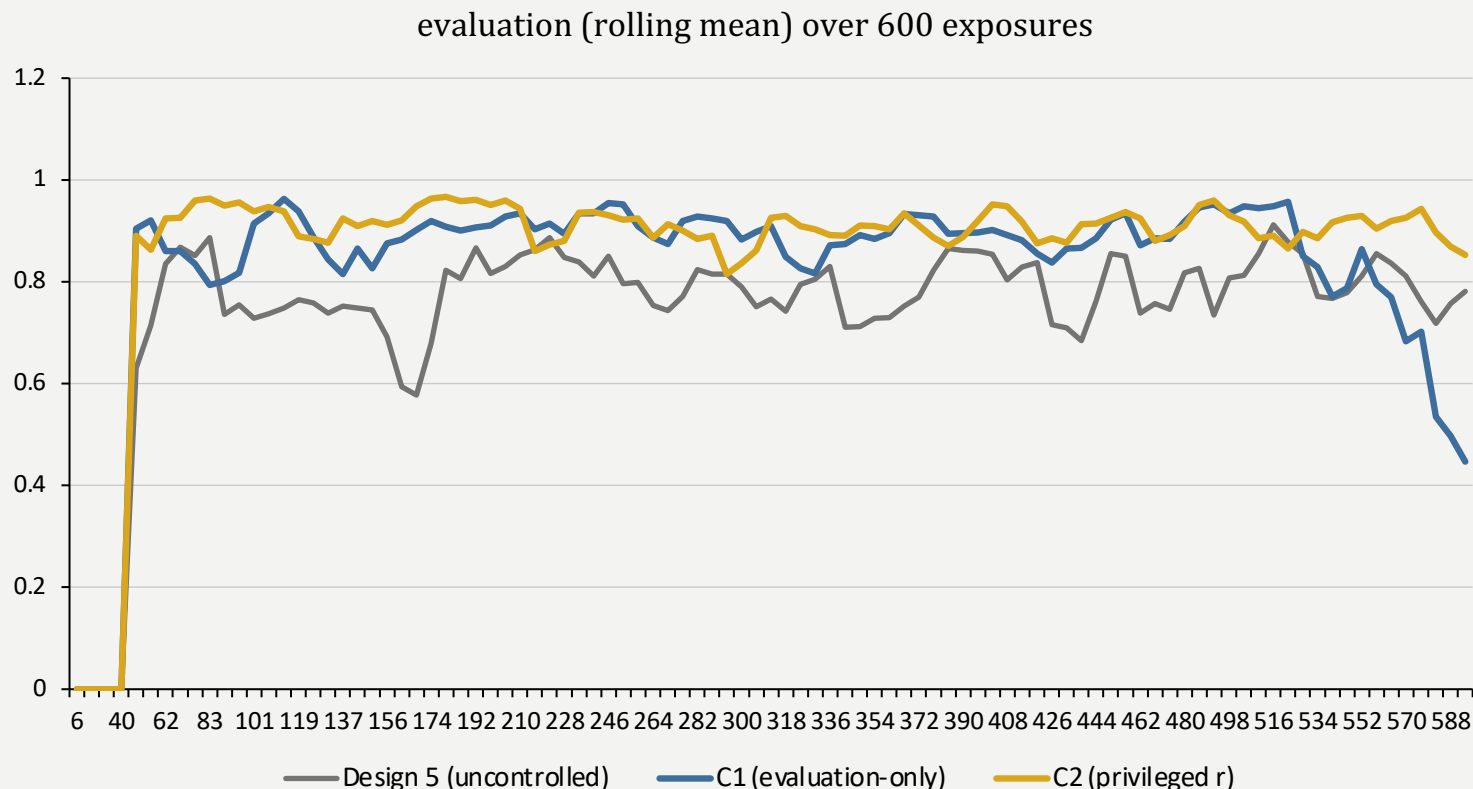
C2 only: `r_change_threshold=0.15`

A DELIBERATE TRADEOFF: NOISE = 0.25

`pyactup.Memory` has no seed for its own activation noise — our seed only fixes song order, so these three runs aren't perfectly reproducible run-to-run. We use `noise=0.25` anyway (the project's real default, `config.yaml`) because realistic run-to-run variation is part of the story we want to show, not something to engineer away.

Strict convergence-trial metric (evaluation ≥ 0.9 sustained) isn't reached by any condition here — median / mean evaluation is the reported comparison instead.

Results: control helps, privileged access helps more



Condition	Median eval.	Mean eval.
Design 5 (random)	0.90	0.79
C1 (evaluation-only)	0.95	0.87
C2 (privileged r)	0.97	0.91

Both controllers beat the uncontrolled baseline on median and mean evaluation, and C2's privileged access widens the gap further still — a clean, monotonic ordering: baseline < C1 < C2. C2 also has the mildest worst-case dip (min evaluation -0.39, vs. -0.96 for C1 and -1.15 for the baseline), consistent with the change-point reset damping out large negative excursions.

Testing this model — and its control

This is a highly theoretical model. Concrete next steps to test it against reality:

Memory prediction-error complexity over atomic symbols is first-order

What about higher-dimensional, realistic stimuli? What about other aspects of aesthetic judgment? Our work is just first-order, simple-as-possible memory approach.

A memory model for recommender systems? Call Netflix?

Are there usable data? What would be the recommendations for such systems?

Could empirically test

Sure, but why?

Implications beyond/outside of domain of aesthetics

Just thought I'd ask. Any ideas?

Open questions welcome.

<https://github.com/mark-orr/aesthetic-memory-model>